شرح المحاضرة 02: التعقيد (Complexity) والـ Array

المادة: DSA27 — Data Structures and Algorithms
الدكتور: Dr. Haitham A. El-Ghareeb
الأسبوع: 2 · Fall 2026

فهرس المحتوى

1. إحنا بنعدّ إيه؟ 1
2. عدّ الـ Loops 2
3. Big-O و Big-Ω و Big-Θ 4
4. Best / Worst / Average 6
5. Time vs Space 7
6. 5 قواعد لقراءة التكلفة من الكود 7
7. الـ Array 9
8. الـ Array بتاعة الكورس 11
9. عمليات الـ Array وتكلفتها 12
10. الـ 2D Arrays 14
11. القاعدة الذهبية 14
14. الملخص 16

1) إحنا بنعدّ إيه؟

طريقتين للإجابة على "الكود ده سريع قد إيه؟"

القياس بالساعة (Measure)	العدّ (Count)
بتشغّل الكود وتقيس الوقت	بتعدّ الخطوات الأساسية
صح على جهازك، وعلى الـ input ده، النهارده بس	صح على أي جهاز
بيديك الثوابت (constants)	بيديك **الشكل** (shape)

ليه العدّ أقوى؟ لأنه بيخليك **تتنبأ**.
لو الخوارزمية بتعمل n^2 خطوة وخدت ثانية عند $n = 10{,}000$، يبقى عند $n = 100{,}000$ (يعني 10 أضعاف) هتاخد حوالي 100 ثانية (لأن n^2 زادت 100 ضعف). على أي جهاز. الستوب ووتش مش هيقولك كده.

محتاج الاتنين: العدّ عشان تفهم، والقياس عشان تتأكد.

نموذج الخطوة الأساسية (RAM Model)

الخطوة الأساسية (basic step) = حاجة بتاخد وقت ثابت مهما كان حجم الـ input:

- عمليات حسابية ومقارنة على أرقام صغيرة: `a + b` , `x < y`
- إسناد قيمة لمتغير: `x = y`
- **قراءة أو كتابة slot واحد** في array: `a[i]` , `a[i] = v`
- استدعاء function أو الرجوع منها (مش اللي بتعمله جواها)

التكلفة = عدد الخطوات كدالة في n (حجم الـ input).

> **نقطة مهمة جداً:** قراءة slot واحد بتكلف خطوة واحدة **مش افتراض مجاني**، دي خاصية مستمدة من طريقة تخزين الـ array في الذاكرة (هنشرحها في قسم الـ Array).

n يعني إيه؟ أي حاجة بتقيس حجم الـ input: عدد عناصر array، عدد حروف string، عدد nodes في graph. وأحياناً في اتنين: matrix حجمها n × m، أو graph فيه V nodes و E edges.

> الـ RAM model تبسيط. الأجهزة الحقيقية فيها cache، وفي Python الـ integers مش ثابتة الحجم. بس التبسيط ده صح لأنه بيحتفظ بالمهم لما n كبير.

---

2) عدّ الـ Loops

Loop عادي

```python
def total(arr):
    result = 0                  # 1 step
    for i in range(len(arr)):   # الـ body بيتنفذ n مرة
        result += arr[i]        #    2 steps: قراءة + جمع وإسناد
    return result               # 1 step
```

$$T(n) = 2n + 2$$

لو عدّيت بدقة أكتر (الـ loop نفسه بيقارن ويزوّد counter) هتطلع 4n + 3 أو 5n + 2. محدش هيتفق على الثوابت، **وده مش مهم**، لأن كل الصيغ دي معناها "ثابت + n × ثابت" وكلها بتكبر بنفس الطريقة.

Nested Loop

```python
def count_pairs(arr):
    n, pairs = len(arr), 0
    for i in range(n):
        for j in range(i + 1, n):       # بيتنفذ n-1, n-2, ..., 1, 0 مرة
            if arr[i] == arr[j]:
                pairs += 1
    return pairs
```

الـ inner loop:

- لما i = 0 بيشتغل n−1 مرة
- لما i = 1 بيشتغل n−2 مرة
- ... لحد 0

المجموع:

$$(n-1) + (n-2) + \ldots + 1 + 0 = n(n-1)/2 = \tfrac{1}{2}n^2 - \tfrac{1}{2}n$$

ده مجموع جاوس المشهور: زاوج أول رقم مع آخر رقم، كل زوج مجموعه n، وعندك (n−1)/2 زوج تقريباً.

الحد الأكبر هو $\frac{1}{2}n^2$. لو n ضعفت، الشغل يزيد **4 أضعاف** تقريباً.

ليه بنرمي الحاجات دي؟

n	$\frac{1}{2}n^2$	$\frac{1}{2}n$	نسبة $\frac{1}{2}n^2$ من الإجمالي
10	50	5	91%
1,000	500,000	500	99.9%
1,000,000	5×10^{11}	500,000	99.9999%

بنرمي حاجتين لسببين مختلفين:

1. **الحدود الصغيرة (Lower-order terms):** بتبطل تفرق. عند n = 1000 الحد $\frac{1}{2}n$ بقى خطأ تقريب.
2. **الثوابت (Constants):** مش خاصية في الخوارزمية. نفس الخوارزمية ليها ثابت مختلف في C وفي Python، على لابتوب وعلى سيرفر. اللي بيفضل ثابت في كل مكان هو **الشكل**: n^2 أو n أو $\log n$.

$$\tfrac{1}{2}n^2 - \tfrac{1}{2}n \longrightarrow n^2$$

الصفقة اللي Big-O بتعرضها: تتنازل عن الثوابت، وتاخد جملة **صح على كل جهاز** وبتقولك إيه اللي يحصل لما n يكبر. صفقة ممتازة لـ n الكبير، ومضللة لـ n الصغير (عشان كده لسه بنقيس).

Big-Θ و Big-Ω و Big-O (3

Big-O: حد أعلى (Upper Bound)

f(n) = O(g(n)) يعني: يوجد ثوابت c > 0 و n_0 بحيث:

$f(n) \le c \cdot g(n)$ لكل $n \ge n_0$

بالعربي: "من نقطة معينة وطالع، f عمرها ما بتكبر أسرع من ثابت "g ×".

فكّك التعريف جزء جزء

الجزء	معناه
"يوجد c و n_0"	أنت اللي بتختارهم. الإثبات معناه إنك تلاقي **زوج واحد** شغّال.
"$c \cdot g(n)$"	الثابت مسموح. كده التعريف بيرمي الثوابت: 1000n هي O(n) لأن c = 1000 شغّال.
"لكل $n \ge n_0$"	بس n الكبير هو اللي بيتحسب. تحت n_0، f ممكن تتصرف زي ما تحب. كده الحدود الصغيرة بتختفي.
"≤"	ده **حد أعلى**: f مش بتكبر **أسرع** من g. مش بيقول إنها بتكبر **بنفس سرعة** g.

نقطة بتلخبط ناس كتير: `n = O(n²)` جملة **صحيحة**! بس مش مفيدة، زي ما تقول "القاعة بتشيل أقل من مليون واحد". لما بنقول "linear search" هي "O(n)" بنقصد بشكل غير رسمي **أضيق** حد أعلى نقدر نثبته.

إثبات واحد (لازم تعرفه)

الادعاء: $3n + 5 = O(n)$

الإثبات: نختار **c = 4** و **$n_0 = 5$**. لكل $n \ge 5$:

```
3n + 5  ≤  3n + n  =  4n
```

(الخطوة دي صح لأن $n \ge 5$، وده بالظبط ليه اخترنا $n_0 = 5$.)

يبقى $3n + 5 \le 4n$ لكل $n \ge 5$. ∎

اختيار تاني شغال برضه: c = 8 و $n_0 = 1$ (لأن $3n + 5 \le 3n + 5n = 8n$ لما $n \ge 1$). التعريف عايز **أي** c و n_0، مش الأحسن.

تعميم: أي polynomial هو Big-O لأعلى أس فيه:
$a_k n^k + \dots + a_1 n + a_0 = O(n^k)$. الطريقة: بدّل كل أس أصغر بـ n^k (لـ $n \ge 1$ كل واحد أصغر)، واجمع المعاملات، والمجموع ده هو c.

إزاي تثبت إن حاجة مش O(g)؟ اثبت إن مفيش c ينفع. مثلاً n^2 مش O(n): لأن $n^2 \le c \cdot n$ معناها $n \le c$ لكل n كبير، ومفيش ثابت أكبر من كل الـ n.

للامتحان: هيجيلك إثبات واحد زي ده، وأسئلة أكتر كتير على "اقرا الـ Big-O من الكود".

Big-Θ g Big-Ω

الرمز	المعنى	زي
f = O(g)	f ≤ c·g في النهاية	≥ حد **أعلى**
f = Ω(g)	f ≥ c·g في النهاية	≤ حد **أدنى**
f = Θ(g)	الاتنين: c₁g ≤ f ≤ c₂g	= حد **محكم** (tight)

- **Ω (أوميجا):** عكس "f .O بتكبر **على الأقل** بسرعة "g.
- **Θ (ثيتا):** الاتنين مع بعض. f محصورة بين مضاعفين ثابتين من g، يعني بتكبر **بالظبط** زي g لحد الثوابت. ده غالباً اللي الناس تقصده لما تقول "Big-O" في الكلام العادي.

مثال: ½n² – ½n هي O(n²) و Ω(n²) وبالتالي Θ(n²). وهي كمان O(n³): صح ومالوش لازمة.

في الكورس وفي الإنترفيوهات: "التعقيد "O(n) معناها Θ(n)" ومحكم". **في الإثبات أو الامتحان** استخدم الرمز اللي تقدر تثبته فعلاً.

استخدام مهم لـ Ω: حد أدنى على مشكلة (مش خوارزمية).
أي خوارزمية ترتيب بتشتغل **بالمقارنة بس** محتاجة Ω(n log n) مقارنة في أسوأ حالة. في Week 9، counting sort بينزل تحتها لأنه **مش بيقارن**، وده بالظبط الثغرة اللي الحد سايبها.

الفئات اللي هتقابلها

Big-O	الاسم	مثال في الكورس	عند n = 1,000,000
O(1)	ثابت (constant)	`a[i]`، push على stack	1
O(log n)	لوغاريتمي	binary search (Week 8)	20
O(n)	خطي	linear search، insert في array	10^6
O(n log n)	linearithmic	merge sort, heap sort (Week 10)	2×10^7
O(n²)	تربيعي	bubble, selection, insertion sort	10^{12}
O(2ⁿ)	أسّي	كل الـ subsets (Week 3)	—

لو الجهاز بيعمل مليار خطوة في الثانية:

- 10^6 خطوة = **ميلي ثانية**
- 2×10^7 = **جزء من خمسين من الثانية**
- 10^{12} = **أكتر من ربع ساعة**
- 2^n عند n = مليون: مفيش رقم مفيد، الكون أصغر من كده.

الـ log n جاي منين؟ من **القسمة على 2 (halving)**. لو قسمت n على 2 مرة ورا مرة لحد ما توصل 1، بتاخد حوالي $\log_2 n$ خطوة. (في Lab 01 قسمنا 100 لحد 1 في 6 خطوات.)

أي خوارزمية بترمي نص الباقي في كل خطوة هي O(log n).

Best / Worst / Average Case (4

تكلفة الخوارزمية **مش دالة واحدة في n**، بتعتمد على **أنهي** input من الحجم n إديتهولها.

```
def find(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

الحالة	امتى	عدد المقارنات
Best	الهدف هو `arr[0]`	1 → O(1)
Worst	الهدف آخر عنصر أو مش موجود	n → O(n)
Average	موجود، وكل مكان احتماله متساوي	(n+1)/2 → O(n)

- **Worst case:** أكبر تكلفة على كل inputs بحجم n. **ده الافتراضي**: لما حد يقول "linear search" هي "O(n) يقصد الأسوأ. وهو **ضمان**: مفيش input يعمل أسوأ.
- **Best case:** أقل تكلفة. نادراً مفيد لوحده. "الـ sort بتاعي O(n) في أحسن حالة" غالباً معناها "لما الـ input مرتب أصلاً" وده مش سبب تختاره.
- **Average case:** التكلفة المتوقعة على **توزيع** معين للـ inputs. **لازم تذكر الافتراض.** في linear search لو الهدف موجود واحتمال أي مكان متساوي: (1 + 2 + ... + n)/n = (n+1)/2. لسه O(n): في المتوسط بتدور في نص الـ array، والنص ده ثابت.

> **تحذير:** متلخبطش بين الاتنين.
> "Worst case" بتقول **أنهي input**. "Big-O" بتقول **أنهي نوع حد**.
> الجملة "الأسوأ في linear search هي Θ(n)" سليمة وبتستخدم الاتنين.

مثال قياس حقيقي

إدخال عنصر في array حجمها 20,000 في 9 مواضع مختلفة:

- عند index 0: كل العناصر لازم تتزحزح (shift).
- عند النهاية: ولا عنصر بيتزحزح.

نفس العملية ونفس n، والفرق آلاف المرات. التكلفة **متناسبة بالظبط** مع عدد العناصر بعد نقطة الإدخال (n – i).
متوسطها على كل المواضع n/2، يعني O(n).

5) Time vs Space

Space complexity: بتكبر إزاي الذاكرة **الإضافية** اللي الخوارزمية محتاجاها مع n.

الدالة	Extra space	ليه
`find(arr, x)`	O(1)	متغيرين مهما كان n
`reverse_in_place(arr)`	O(1)	swaps جوه نفس الـ array
`resized(arr, 2n)`	O(n)	array جديدة كاملة
`merge_sorted(a, b)`	O(n + m)	الـ output array
`total(values)` recursive	O(n)	stack frame لكل عنصر

- بنعدّ **auxiliary space**: الذاكرة **بره** الـ input نفسه.
- **الصف الأخير هو اللي الناس بتنساه: الـ Recursion بتستهلك ذاكرة!** كل استدعاء لسه ما رجعش بيمسك frame على الـ call stack. دالة recursive بعمق n بتاخد O(n) من الـ stack حتى لو مبتنشئش data. Pythong بيوقفها عند حوالي 1000 frame.
- **Time و Space بيتبادلوا:** merge sort بيشتري O(n log n) وقت بـ O(n) مساحة إضافية؛ heap sort بيجيب نفس الوقت بـ O(1) مساحة ويدفع تمن في حاجات تانية.

6) خمس قواعد لقراءة التكلفة من الكود

#	القاعدة	الخلاصة
1	**التتابع (Sequence) ← جمع**	O(f) وبعدها O(g) = O(f + g) **الأكبر فيهم**
2	**Loop ← ضرب**	O(g) = O(n·g) بـ n iteration × body
3	**Nested loops ← ضرب تاني**	loopين كاملين على n = O(n²)
4	**Halving ← لوغاريتم**	loop بيقسم الباقي على 2 = O(log n)
5	**الاستدعاء بيكلف قد الدالة**	بما فيها الـ built-ins

تفاصيل كل قاعدة

قاعدة 1: loop من O(n) وبعده loop من O(n²) = O(n + n²) = O(n²) **O(n²)**. الحد الأكبر يبلع الأصغر.

قاعدة 2: لازم تنتبه **الـ body بيكلف كام**. Body من 3 خطوات O(1) =، فـ n iteration = O(n). بس body بيعمل O(n) شغل (قاعدة 5) يخلي الـ loop O(n²).

قاعدة 3: الـ triangular loop (اللي `j` فيه بيبدأ من `i + 1`) لسه O(n²): بيعمل نص شغل الـ square loop، والنص ثابت.

قاعدة 4: من هنا بييجي log n:

```
steps = 0
while n > 1:
    n //= 2
    steps += 1          # بيتنفذ حوالي log2(n) مرة
```

قاعدة 5: دي اللي بتصيد مبرمجين Python، وعشانها نص المحاضرة التاني موجود.

قاعدة 5 بالتفصيل: الـ Hidden Loop

```
def has_duplicate(values):
    for i in range(len(values)):
        if values[i] in values[i + 1:]:      # <- السطر ده بيكلف قد إيه؟
            return True
    return False
```

سطر واحد، **و loopاين مخفيين من O(n)**:

- `values[i + 1:]` (slice) **بتنسخ** لحد n عنصر ← O(n)
- `in` **بتدوّر** عليهم واحد واحد ← O(n)

جوه loop من n iteration = **O(n²)**.

> **استدعاء مش O(1) إلا لو عارف إنه O(1).**

حاجات شكلها خطوة واحدة وهي لأ:

العملية	التكلفة	السبب
`values[i+1:]`	O(n)	بتبني list جديدة وتنسخ
`x in some_list`	O(n)	بتمشي وتقارن
`list.insert(0, x)`	O(n)	بتزحزح كل العناصر
`list.pop(0)`	O(n)	بتزحزح كل العناصر
`list.remove(x)`	O(n)	بتدوّر وبعدين بتزحزح
`list.index(x)`	O(n)	بتدوّر
`sorted(xs)`	O(n log n)	ترتيب كامل

Python built-ins متهندسة ممتاز وتكلفتها موثقة، بس هي **مخفية** جوه سطر قصير. **متقدرش تعدّ تكلفة كود مش شايفه.** عشان كده من النهارده هتشوفها.

---

7) الـ Array

إيه هو؟

عدد ثابت من **خانات (slots) متساوية الحجم**، متخزنة **جنب بعض** في الذاكرة.

```
address of a[i] = base + i × slot_size
```

ضرب واحد وجمع واحد **مهما كان i ومهما كان n**. عشان كده `a[i]` هي **O(1)**.

كل حاجة عن الـ array بتطلع من الصورة دي

1. الـ indexing O(1) جاية من خاصيتين مع بعض:

- الخانات **متجاورة (contiguous)**: مفيش فجوات، كتلة واحدة
- الخانات **متساوية الحجم**

لو شلت أي واحدة، الـ indexing بيبطل O(1). مثلاً الـ linked list (Week 5) بتحط عناصرها في أي مكان في الذاكرة، فالوصول للعنصر i معناه تمشي i رابط.

2. الحجم الثابت هو التمن. الكتلة اتحجزت بحجم معين، والذاكرة اللي بعدها ممكن تبقى مملوكة لحاجة تانية. الـ array **مش بتكبر في مكانها**. عشان تاخد أكبر لازم تحجز كتلة جديدة وتنسخ: O(n). ده كل Week 4.

3. ليه الـ index بيبدأ من 0؟ لأن i هو **offset** من الـ base: العنصر 0 على بعد صفر خانات من البداية. اللغات اللي بتبدأ من 1 لازم تطرح 1 في مكان ما.

> **مصطلحات بالعربي:** Array = المصفوفة · Contiguous memory = ذاكرة متجاورة

في C: الـ Array هو الماكينة نفسها

```c
int a[6] = {17, 4, 42, 8, 15, 23};   // اتحجزت مرة واحدة، 6 × 4 bytes
a[2] = 99;                           // تعليمة واحدة: base + 2 × 4
a[6] = 0;                            // بتكتب فوق أي حاجة جنبه — مفيش check
```

- الحجم جزء من الـ type، ثابت وقت الـ compile
- كل slot فيه `int` يعني 4 bytes
- **مفيش bounds check**: index برّه المدى مش error، ده **bug**

السطر الأخير مشهور: `a[6]` بتحسب base + 6×4 وتكتب هناك، فوق أي متغير ساكن جنبه. ده أصل عيلة كاملة من الثغرات الأمنية اسمها **buffer overflows**. كل array في الكورس **بيعمل check** ويرمي `IndexError` : المقارنة الزيادة O(1) وتستاهل.

الـ Arrays في Python: 4 مرشحين

النوع	بيكبر؟	بيشيل	بيخفي التكلفة؟
`list`	أيوه (dynamic array)	references لأي objects	**أيوه**: `append` , `insert` , `pop(0)` , `in` , slicing
`array.array`	أيوه	أرقام بس، نوع واحد، packed	أيوه: نفس methods الـ list
`numpy.ndarray`	لأ	أرقام، نوع واحد، packed	أيوه: عمليات على الـ array كله في C
`ctypes` array	**لأ**	قيم C أو references	**لأ**: index و assign وبس

- **`list` :** هي نفسها **dynamic array**، اللي هتبنيها في Week 4. جواها كتلة capacity + size + references. كل method ليها خوارزمية بتكلفتها، مكتوبة جاهزة.
- **`array.array` :** بتخزن الأرقام متضغطة (4 bytes للعنصر بدل reference لـ int object). مدمجة ومفيدة للبيانات الرقمية الكبيرة، بس تقدر `append` و `insert` و `pop` زي list، يعني dynamic برضه.
- **NumPy:** fixed-size ومتضغطة، عمود الـ data science. قوتها في العمليات الكاملة (`a + b` بتجمع مليون رقم في سطر)، ممتازة للحساب، **مالهاش لازمة** لو عايز تفهم تكلفة إدخال واحد.
- **`ctypes` :** arrays C حقيقية. حجم ثابت، تقدر تعمل index و assign بس. **ده بالظبط الـ primitive اللي إحنا عايزينه.**

Python بيخزن References

الـ slot في array بتاعة Python **مش بيحتوي** `42` ، بيحتوي **reference للـ object** `42` . الخانات متساوية (reference واحد لكل واحدة) حتى لو الـ objects مش متساوية. كده array واحدة تشيل int و string و list، ولسه بتعمل index في O(1).

- حجم الـ slot = حجم reference واحد (8 bytes على 64-bit)، فحساب العنوان شغال.
- **التمن: قفزة تانية.** `a[i]` بتلاقي الـ reference في O(1)، وبعدين تتبعه للـ object اللي ساكن في أي مكان حطه فيه Python.
- الـ packed arrays (`array.array` , NumPy) بتتجنب القفزة دي بتخزين الأرقام نفسها، عشان كده أسرع للشغل الرقمي، وعشان كده بتشيل نوع واحد بس.

8) الـ Array بتاعة الكورس (`dsa/array.py`)

```
from dsa.array import Array

a = Array(5)                    # 5 خانات، كلها None. ثابتة للأبد.
a[0] = 42                       # O(1)
a[0]                            # O(1) -> 42
len(a)                          # 5: الـ capacity، مش "كام واحدة مستخدمة"
list(a)                         # iterate: [42, None, None, None, None]

Array.from_values([3, 1, 2])    # array ممتلئة، للـ tests والـ REPL
Array(3, fill=0)                # كل خانة تبدأ بـ 0
```

دي **C array حقيقية من references** (`ctypes`) ملفوفة بحيث **تعمل bounds check**. الملف جاهز، اقراه وماتعدلوش.

3 تفاصيل مهمة

1. `len(a)` **هي الـ capacity.** `Array(5)` طولها دايماً 5 سواء حطيت فيها حاجة ولا لأ. تتبع **كام خانة مستخدمة (الـ size)** هو **شغلك أنت** في متغير منفصل. **الفصل ده (capacity ضد size) هو قلب الـ dynamic array.**
2. **كل خانة بتبدأ** `None` (أو الـ `fill`). خانة ctypes خام بتبدأ NULL، وقراءتها بترمي `ValueError` مش واضح، فالـ class بيملاها.
3. `from_values` بتنسخ iterable لـ Array جديدة ممتلئة تماماً. للـ test data والتجارب في الـ REPL. **متستخدمهاش جوه structure عشان تهرب من القواعد**: التحويل لـ tuple والرجوع هو بالظبط الـ hidden O(n) اللي المحاضرة بتحذر منه.

اللي الـ Array مش هتعمله

```
>>> a = Array(3)
>>> a[3]
IndexError: Array index 3 out of range for length 3 ...
>>> a[-1]
IndexError: ...                      # مفيش negative indices زي C
>>> a[0:2]
TypeError: Array does not support slicing; copy with a loop
>>> a.append(1)
AttributeError: 'Array' object has no attribute 'append'
```

مفيش grow، shrink، insert، slicing، ولا search methods. **كل واحدة من دول خوارزمية، وإنت اللي هتكتبها.**

كل رفض مقصود:

الرفض	السبب
مفيش negative indices	الـ index هو offset من الـ base ومفيش حاجة قبل الـ base. في Week 4 هتنفّذ `arr[-1]` في `DynamicArray` وهتشوف إنه **ترجمة** `i + size` فوق array مفيهاش ده.
مفيش slicing	الـ slice **نسخة** O(k) متنكرة في شكل خطوة واحدة. لو عايز نسخة اكتب الـ loop وشوف التكلفة.
مفيش append	الـ array مبتكبرش. ده تعريفها.

---

9) عمليات الـ Array وتكلفتها

العملية	إزاي	التكلفة
قراءة/كتابة slot `i`	حساب عنوان	**O(1)**
إيجاد قيمة	فحص الخانات واحدة واحدة	**O(n)**
insert عند `i`	زحزح `i .. size-1` **يمين** وبعدين اكتب	**O(n)**
remove عند `i`	زحزح `i+1 .. size-1` **شمال**	**O(n)**
insert/remove **في النهاية**	مفيش حاجة تتزحزح	**O(1)**
grow	array جديدة، انسخ كل حاجة	**O(n)**

الجدول ده هو **عقد الـ array** (الـ ADT بتاعها بالتكاليف). كل structure هتبنيها الترم ده **بترث التكاليف دي** من الـ array اللي تحتها.

صفّين النهاية بيحددوا حاجات كتير

الإدخال والحذف في **النهاية** O(1) لأن مفيش حاجة بتتحرك. الحقيقة دي بتحدد:

- إن الـ **stack** لازم يحط الـ top في **نهاية** الـ array (Week 6)
- إن الـ **queue** مينفعش تشيل من **أول** الـ array ببساطة، وعشان كده Week 7 محتاجة ring buffer
- إن الـ **heap** بتضيف في النهاية وتصلّح لفوق (Week 12)

الإدخال slot بـ slot

```python
def insert_at(arr, size, index, value):
    for i in range(size, index, -1):    # من النهاية، للخلف
        arr[i] = arr[i - 1]
    arr[index] = value
    return size + 1
```

دي قلب `insert_at` بدون error checks (العقد الكامل في الـ docstring: `OverflowError` لو ممتلئة، `IndexError` لو index غلط).

ليه للخلف؟

لو زحزحت للأمام (نسخت `arr[2]` في `arr[3]` الأول) هتكتب فوق `d` قبل ما تحركه.

مثال: `arr = [a, b, c, d, _]` وعايز تدخل `X` عند index 2.

الطريقة	الخطوات	النتيجة
صح (من اليمين)	`arr[4]=d` ثم `arr[3]=c` ثم `arr[2]=X`	`[a, b, X, c, d]` ✓
غلط (من الشمال)	`arr[3]=arr[2]=c` (ضاعت d!) ثم `arr[4]=arr[3]=c`	`[a, b, c, c, c]` ✗

> **القاعدة:** الزحزحة **يمين** ← ابدأ من **اليمين**. الزحزحة **شمال** (الحذف) ← ابدأ من **الشمال**.
> تتبّع الاتنين على ورق مرة، **ده أشهر bug في تمارين الأسبوع**.

الـ loop بيشتغل `size - index` مرة:

- index 0: `size` حركة (أسوأ حالة)
- index = size: صفر حركة (يعني كتابة في النهاية بس)

القياس الحقيقي

تلات عمليات على الـ `Array` بتاعة الكورس، n بيتضاعف من 1,024 لـ 32,768 (المحورين logarithmic: الخط المستقيم الطالع O(n) =، والخط المسطح O(1) =):

العملية	النتيجة
Indexing	ثابت حوالي ربع ميكروثانية مهما كان n (الرسمة بتقيس 10,000 وتقسم لأن الوصول الواحد أسرع من إنه يتقاس)
Linear search لقيمة مش موجودة	بتتضاعف لما n يتضاعف ← O(n)
Insert عند index 0	بتتضاعف برضه ← O(n)، **وفوق الـ search** لأن كل خطوة فيها قراءة **و** كتابة

> **نفس الـ Big-O، ثابت مختلف:** بالظبط اللي Big-O بترميه، وبالظبط اللي القياس بيوريهولك.

عند n = 32,768، الـ index الواحد أرخص من الـ front insertion بحوالي **25,000 مرة**. ده الفرق بين O(1) و O(n).

10) الـ 2D Arrays (Row-Major)

الذاكرة 1D، فالـ array ثنائية لازم تتفرد في كتلة واحدة، صف ورا صف.

```
index of (r, c) = r × cols + c
```

مثال: matrix 3 صفوف × 4 أعمدة. العنصر (2, 1) مكانه: `2 × 4 + 1 = 9` .

- **Row-major** (الصف 0 بعده الصف 1 و NumPy افتراضياً C ...):
- **Column-major:** Fortran و MATLAB
- ده برضه حساب عنوان، فـ (r, c) لسه **O(1)**

البديل: array of arrays (كل خانة فيها `Array` تانية). شغالة، وتكلف قفزة زيادة لكل وصول، وبتسمح بصفوف مختلفة الطول. دالة `transpose_flat` في التمارين بتستخدم الـ flat layout عشان تشتغل على حساب الـ index بإيدك.

---

11) القاعدة الذهبية من النهارده

ممنوع `list` أو `dict` أو `set` جوه structure

أي data structure في `dsa/` بتخزن بياناتها **بس** في:

1. الـ `Array` بتاعة الكورس (`dsa/array.py`)، أو
2. **node objects:** `Node` , `TreeNode` , `Entry` , ...، أو
3. **structure تانية بنيتها قبل كده** في `dsa/` .

أبداً في `list` أو `dict` أو `set` أو `collections.deque` .

الـ lists **لسه مسموحة كـ interface**: الـ functions تاخد list وترجّع list، والـ tests بتستخدمها براحتها.

الـ Structures بتتراكم أسبوع ورا أسبوع

Week	Structure	بتخزن data في
4	`DynamicArray`	`Array` ، بتتبدل بأكبر لما تمتلي
5	`LinkedList`	`Node` objects
6	`Stack`	الـ `DynamicArray` بتاعتك، الـ top في النهاية
7	`SlowQueue` / `CircularQueue`	`Array` / `DynamicArray` كـ ring
11	`BinarySearchTree`	`TreeNode` objects
12	`MinHeap` , `PriorityQueue`	`MinHeap` / `DynamicArray` بتاعتك
13	`ChainingHashMap` / `OpenAddressingHashMap`	`Array` من chains من `Entry` / اتنين `Array`
14	`Graph` / `MatrixGraph`	`DynamicArray` s بتوعك g hash map

في Week 14 الـ graph مبنية على الـ hash map بتاعتك، المبنية على الـ array بتاعتك؛ والـ BFS بتشتغل على الـ queue بتاعتك. **مفيش حاجة مستلفة.** لما حاجة تبقى بطيئة هتعرف أنهي سطر في كودك **أنت** خلّاها بطيئة.

ليه مش lists؟

مش لأن الـ lists وحشة (دي ممتازة وهتستخدمها طول حياتك)، لكن لأنها **بتخبي ورا method مريحة بالظبط التكلفة اللي الكورس موجود يعلّمها.** stack مبنية على `list.append` مش هتعلمك ليه الـ stack O(1). stack مبنية على array كبّرتها بإيدك هتعلمك.

المسموح والممنوع

✅ مسموح	❌ ممنوع
Python lists كـ **inputs g return values** (`bfs` بترجّع list من nodes)	`list` / `dict` / `set` كـ **storage** لـ structure
متغيرات محلية عادية	`list` كـ working storage لخوارزمية عندها structure بتاعتك (استخدم `CircularQueue` في BFS، مش list)
Tuples لإرجاع قيمتين	`from_values` جوه structure للهروب من القواعد

انتبه: الـ TAs هيتأكدوا من ده وقت التصحيح. **structure عدّت كل الـ tests وهي مخبية `list` جواها = تتحسب "مش مكتملة".**

شجرة "مبني على"

```
Array                          (given, O(1) index, fixed size)
 └── DynamicArray              (append amortised O(1) بتكبر بالنسخ، Week 4:)
      ├── Stack                (Week 6)
      ├── SlowQueue            (O(n) هي Week 7 — pop(0)، قيسها)
      └── MinHeap              (Week 12)
           └── PriorityQueue
 ├── CircularQueue             (ring على الطرفين، Week 7 — O(1))
 └── hash maps                 (Week 13)
      └── Graph                (DynamicArray مع ,Week 14)
```

كل سهم = "مبني على". الـ linked lists والـ trees جنب الشجرة دي، مبنية من nodes مش arrays، وWeek 5g موجودة عشان تقارن العيلتين مباشرة.

---

14) الملخص: سبع حاجات تحفظها

1. عدّ **الخطوات الأساسية** كدالة في n؛ احتفظ بـ **الشكل**، ارمي الثوابت والحدود الصغيرة.
2. f = O(g): **في النهاية** f ≤ c·g. Ω هي ≤؛ Θ الاتنين.
3. **Worst case** هو الضمان. قول أنهي حالة تقصد.
4. **التتابع يجمع، الـ loops بتضرب، الـ halving هو log n**، والاستدعاء بيكلف قد الدالة.
5. الـ array = **خانات متجاورة متساوية**: index بـ O(1)، **حجم ثابت**، searchg insert بـ O(n).
6. الـ `Array` بتاعة الكورس: index و assign و length. ومفيش غير كده.
7. من النهاردة: الـ structures بتخزن data في `Array` **أو nodes أو structures بتاعتك، أبداً** `list` أو `dict` أو `set`.